



第9章：知识编辑与模型鲁棒性

- 1) Training-free Knowledge Update
- 2) Jailbreak Robustness
- 3) Steganography & Detection



学习目标与主线

01

不重新训练如何纠错/更新知识（有哪些路线）

02

越狱与注入为何在长上下文/多模态时代更危险

03

隐写/水印：攻击与防御的同一“嵌入”思想



不重新训练条件下的知识修正 ——知识编辑

定义与边界

知识编辑与微调（如LoRA）、RAG和in-context提示等相邻方法的区别与适用范围

01 Model Editing (知识编辑):

对特定事实做局部修正，追求 reliability + locality + portability

02 PEFT/LoRA:

仍是训练（梯度），多为分布级适配

03 RAG:

不改模型，把新知识放外部检索

04 In-Context Editing/IKE:

不改参数，把“新知识+示例”放上下文（非持久）

方法	是否训练	是否持久	典型风险
Model Editing (KE)	否 (直接改参数/权重局部更新，非梯度训练)	是 (写入模型参数)	局部性/连带影响（副作用）； 多编辑冲突； 可逆性与验证成本
PEFT/LoRA	是 (梯度训练，低秩适配)	是 (适配权重持久，需加载)	分布漂移； 过拟合/遗忘； 版本管理与部署复杂度
RAG	否 (不改模型参数)	否 (知识在外部库，可更新但不写入模型)	检索失败/召回偏差； 上下文注入与幻觉； 延迟与依赖外部系统
In-Context Editing (IKE)	否	否 (仅当前上下文生效)	上下文长度限制； 提示脆弱性； 可迁移性差/不稳定

知识编辑的形式化问题与约束

Edit Request

编辑输入

(s, r, o)



三元组形式
(Knowledge
Triplet)



编辑目标

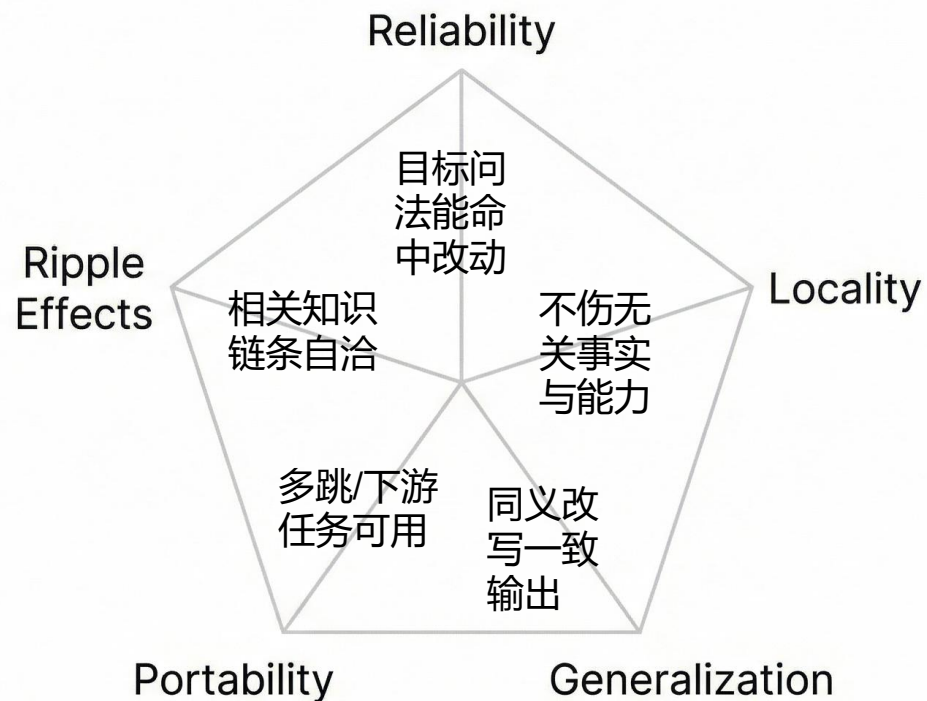
(s, r, o^*)

- s = subject (主体/实体)
- r = relation (关系)
- o = object (客体/属性值/答案)

对同一个**主体 s** 和**关系 r** , 把模型原来认为的答案 o 改成**新的答案 o^*** , 并且尽量**只影响**这条目标事实及其应当影响的推论。

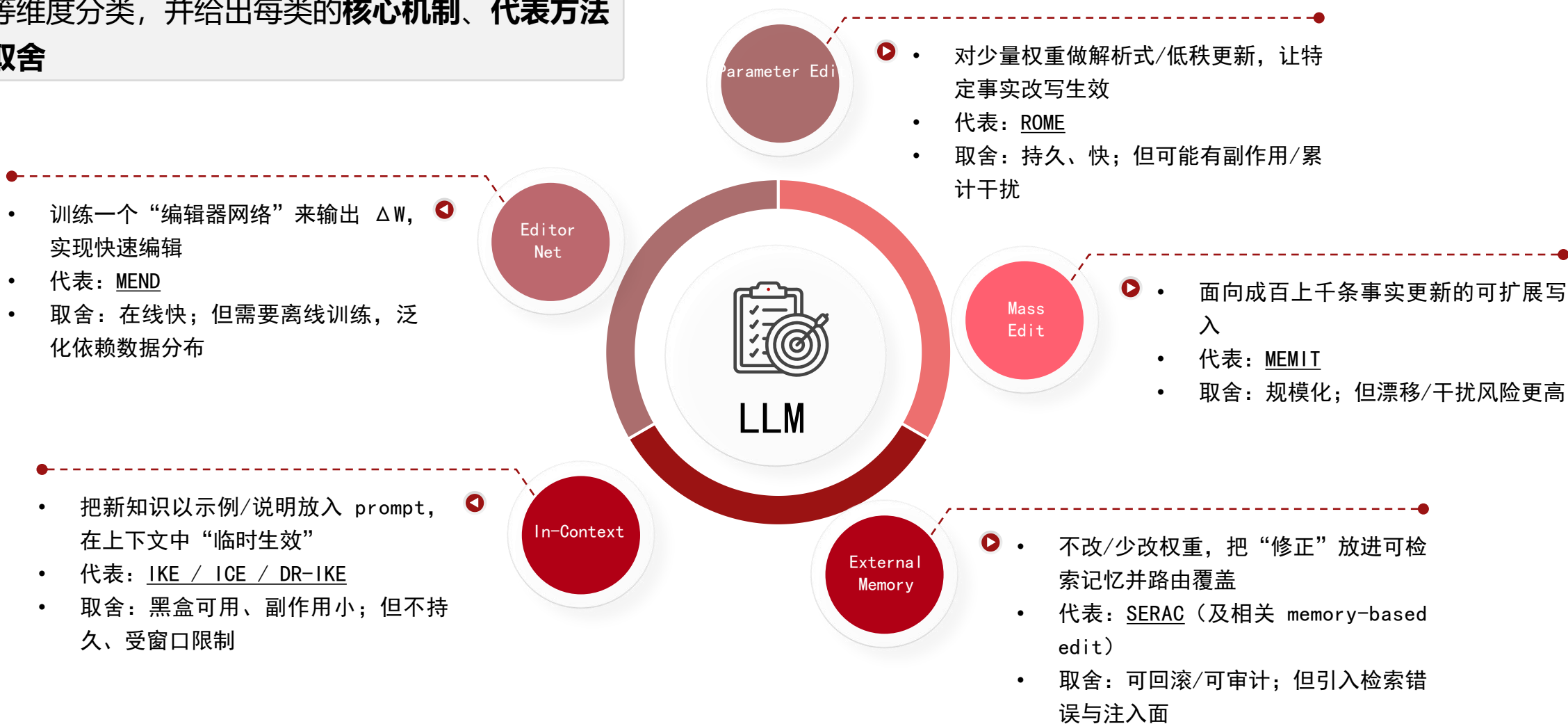


Constraints



方法谱系总览—主流做法

在已确定的知识编辑范围内，把主流解法按更新载体与持久性等维度分类，并给出每类的**核心机制**、**代表方法**与**典型取舍**



ROME: Locate-then-Edit 的核心思想

关键流程

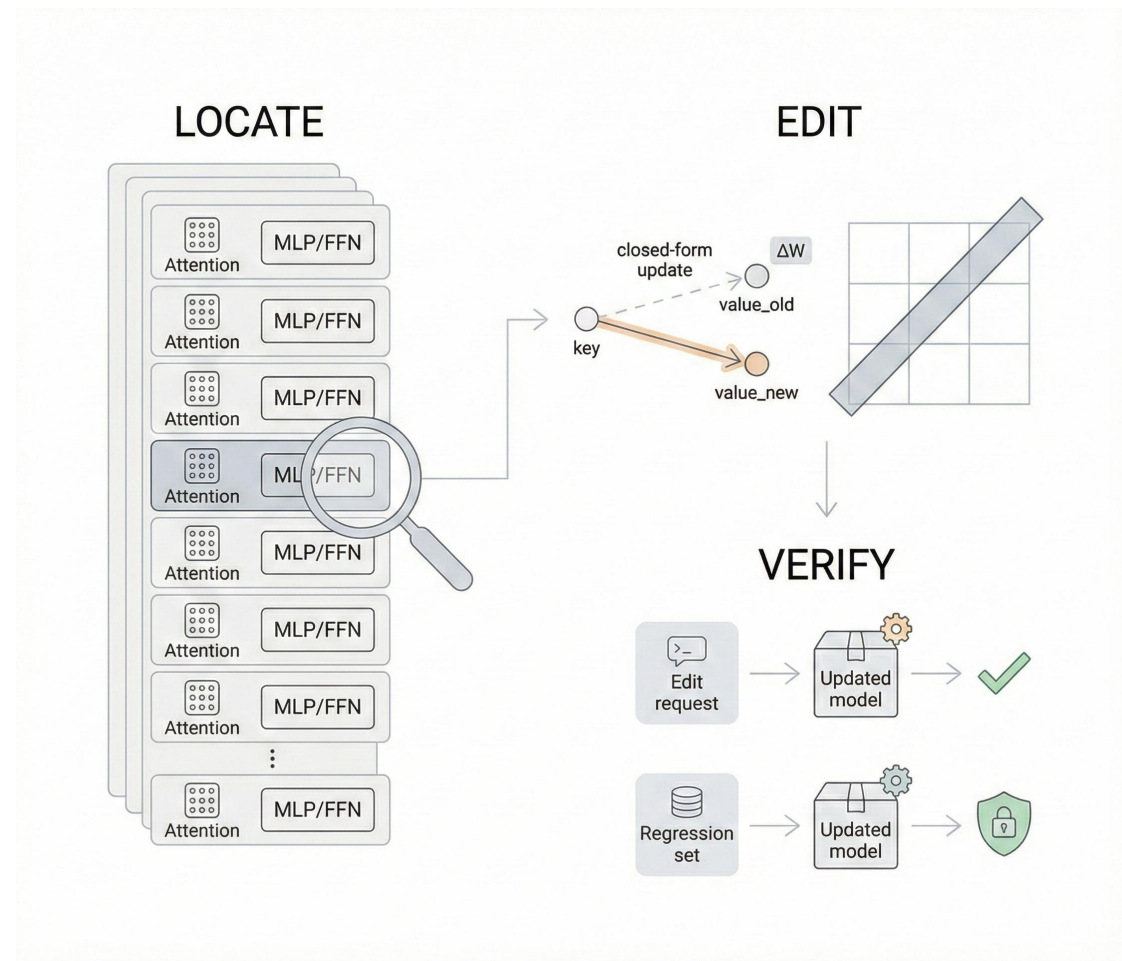
先定位-再编辑

- 先定位：哪个层/哪个 MLP 承载该事实关联（causal tracing/激活追踪）
- 再编辑：把事实看成 $\text{key} \rightarrow \text{value}$ 关联，用解析式构造对 MLP 权重的低秩/定向更新，使输出更偏向新 value

优点缺点

优点-缺点

- 优点：单点编辑快、可控
- 缺点：定位不一定可靠、可能产生副作用（与后续工作对“定位是否真有效”的质疑可点到）



MEMIT: Mass Editing / 可扩展性



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

MEMIT出现的必须

- **性**线上知识更新不是“单次修补”，而是持续流入的变更流
- 单事实编辑（如 ROME）难以规模化
- 批量更新存在“共享结构”更理想的是用一种机制把多条编辑在同一套层/子空间里组织起来

MEMIT 在做什么？

- 把“单条事实写入”升级为“多条记忆的结构化注入”
- 共享定位：用少量关键层承载大批编辑
- 共享写入：用结构化（低秩/解析式）更新同时满足多条约束
- 面向“编辑吞吐量”的工程目标

为什么批量更难？

- 干扰累积与漂移 (Interference / Drift)
- 局部性变差 (Locality degradation)
- Portability 更难：跨表述与跨任务一致性下降
- Ripple Effects 更难：逻辑连锁一致性不足

为什么需要

核心思路

常见问题

MEND: Editor Network

机制

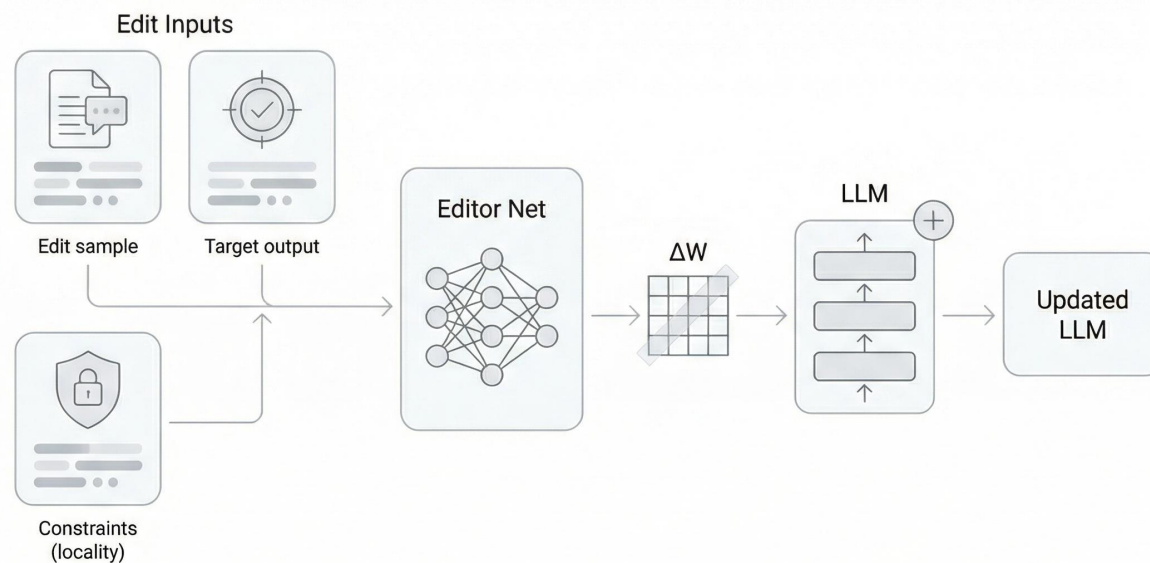
目标-输入-输出-训练

- **核心目标：**把“编辑”从一次次手工/解析更新，变成可学习的通用操作
- **输入信号：**用少量监督定义“这次编辑想达到的行为”
- **编辑器网络输出：**结构化的参数增量 ΔW （通常是低秩/局部化更新）
- **训练方式：**离线训练编辑器，使其在不同编辑任务上泛化

优点 缺点

优点-缺点

- **优点：**推理期编辑速度快、更新更“可控/可约束”、可做“批处理式在线编辑”、可以融入部署流程
- **缺点：**需要离线训练编辑器（数据与成本）、跨模型/跨架构迁移有限、评测“看上去很好”但指标分布与真实分布不一致



编辑器网络将编辑目标直接映射为结构化参数补丁 ΔW ，实现快速且尽量局部的知识更新。

SERAC: 外部记忆/检索式编辑 (半参数)

思路

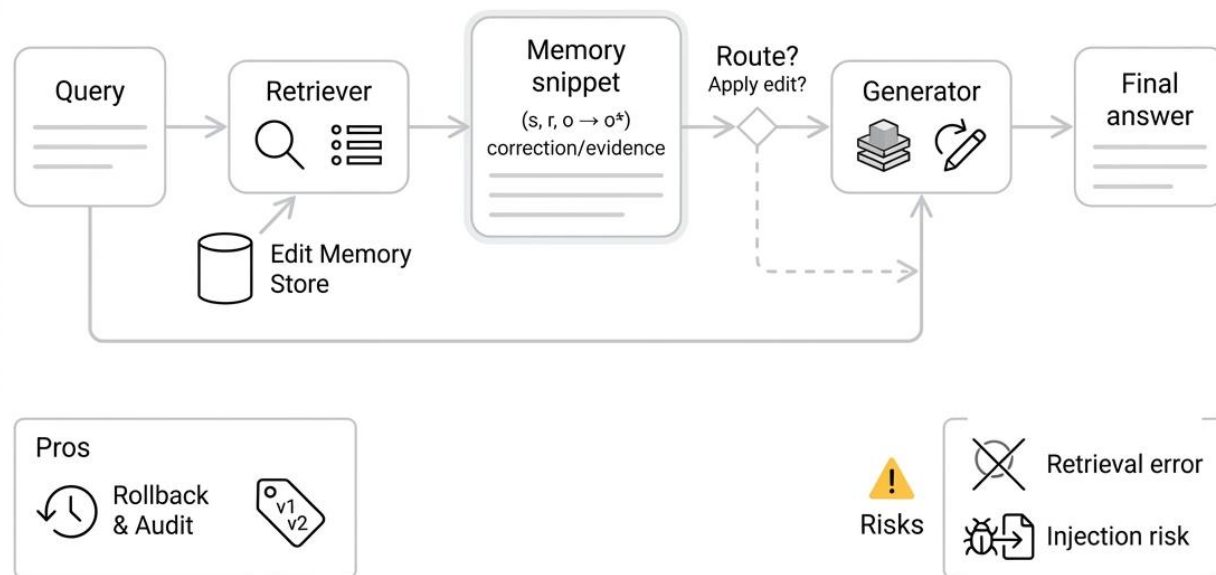
- 把“编辑”从参数更新转为“记忆覆盖”
- 记忆内容通常是“反事实/纠错对”或“编辑示范”
- 推理时采用“检索 + 路由/判别 + 生成重写”的流水线
- 可以理解为“RAG 的一种特化”：检索的不是外部百科，而是“修正补丁库”

优点

- 优点：**可回滚、可审计（治理友好）/对频繁变更更稳，避免参数层面的“累计干扰”/黑盒/跨模型适配性更强/容易做“范围控制”与“分场景生效”

缺点

- 缺点：**检索质量决定上限，容易出现检索错就“改错地方”/“新幻觉”与“证据污染”风险/引入更大的攻击面——外部记忆与检索链路是天然的注入入口



SERAC 将知识修正外置为可检索的“编辑记忆”，在推理时按需路由注入以覆盖答案，但其效果高度依赖检索质量并引入注入风险。

IKE / In-Context Knowledge Editing

核心思路

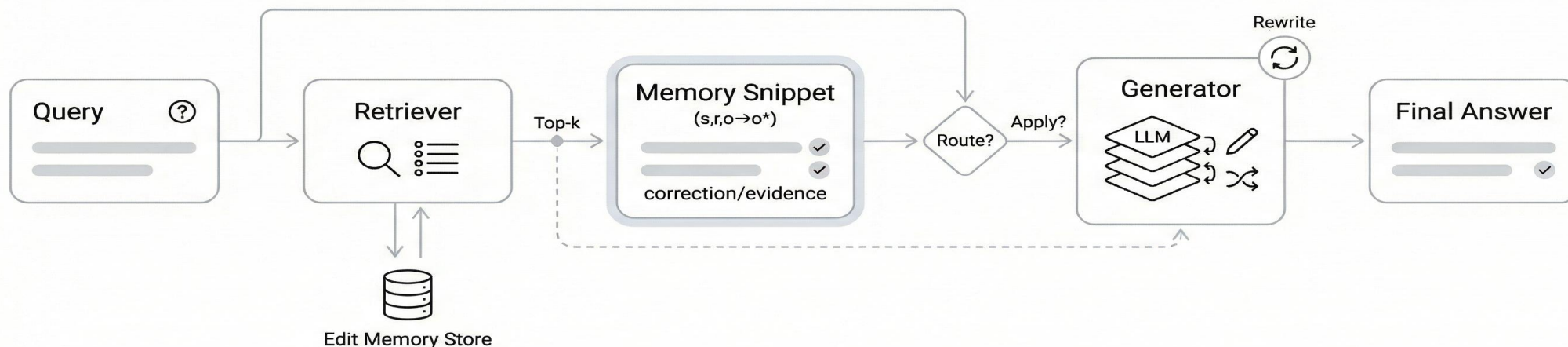
不改动模型参数、也不做梯度训练，而是把“新知识”以少量高质量的上下文示例（demonstrations）写入当前 prompt，让模型在本次推理中通过 in-context learning 临时采纳这条更新后的事实；通常会用“原事实→新事实”的对照示例、同义改写问法和约束性说明来强化一致性，使模型在相关提问上稳定输出新答案。

优缺点

优点：黑盒可用、部署成本低且可快速迭代：无需访问权重或训练数据即可实现纠错，移除示例即可回滚，且由于不持久写参数，往往副作用更小、误伤范围更可控；

缺点：效果不持久、强依赖上下文窗口与提示预算，在长对话或复杂任务中容易被稀释或被其他指令覆盖，同时示例本质属于外部输入，若输入链路不安全，容易受到 prompt injection/间接注入污染而导致错误触发或被恶意引导。

流程



评测体系：从“改对”到“改得干净、改得一致”



Metric	What it tests	Example query type	Common failure	Representative benchmark
Reliability (Efficacy)	目标事实是否被“改对”，在目标关系/问法上稳定输出新答案 o*	直接问目标三元组/关系： e.g., “The capital of X is ?”	改不动； 只在某种问法下改动； 解码/措辞变化时回退旧答案	CounterFact / zsRE； 典型方法：ROME, MEMIT, MEND
Locality (Specificity)	编辑是否“改得干净”：不相关知识与能力不退化，副作用范围受控	无关事实问答；近邻实体/关系问答； 编辑主体出现但不应触发的场景	“误伤/串改”近邻事实； 出现 loud facts； 泛化成全局偏置	CounterFact(含 neighborhood)； CounterFact+（更容易暴露副作用）； 典型方法：ROME, MEMIT, MEND
Generalization	编辑是否“改得一致”：同义改写、不同模板、不同上下文表达下仍保持新知识	Paraphrase / template variants / 多种自然语言表述	只记住某个模板； 换句式就失效； 与原知识并存导致自相矛盾	CounterFact(Paraphrase/Generalization prompts) / zsRE(Rephrases)； 典型方法：ROME, MEMIT, MEND； 对比基线：ICE/IKE
Portability	编辑能否跨“任务形态/上下文”迁移： QA→对话→事实核验等不同输入包装仍生效	同一事实以对话轮次、事实核验声明、长上下文引用等形式出现	在新任务格式下失效； 需要特定触发前缀； 上下文干扰导致覆盖或冲突	SERAC 的跨任务编辑设定（QA / Fact-checking / Dialogue）； 典型方法：SERAC； 对比：参数编辑（ROME/MEMIT/MEND）
Ripple Effects	编辑后的知识能否与相关知识“逻辑连锁一致”：组合推理、多跳关系、派生事实保持协调	需要推理/组合的查询：如“若 X 的球队变了，则其队友/主场/联赛等派生问题？”	能改对单点，但推理链/派生事实仍沿用旧世界； 逻辑不闭合、前后矛盾	RippleEdits（专测 ripple effects/逻辑连锁）； 结论常见：主流参数编辑普遍失败，简单 ICE 基线反而更强

现实难点与研究新趋势



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

现实难点

多跳推理/组合
知识一致性差

连续更新下的
遗忘与干扰

评测与真实应用落差
("编辑在QA场景更
难"等)

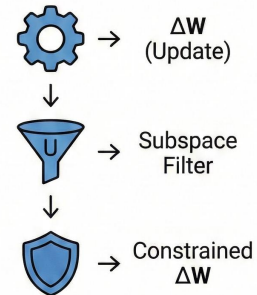
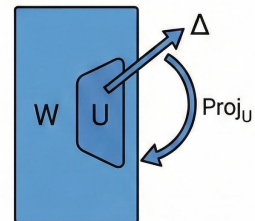
研究新趋势

子空间约束

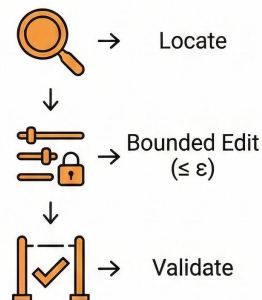
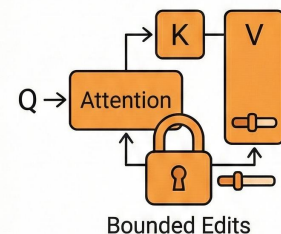
加保守的KV编辑

面向真实QA的评测重构

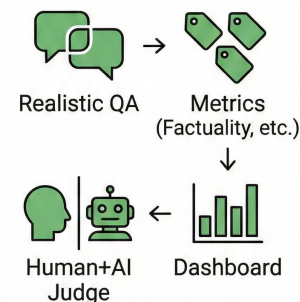
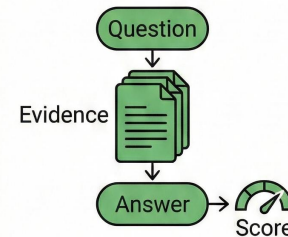
Subspace Constraint



Conservative KV Editing



Evaluation Rebuild





抵御越狱攻击 (Jailbreak) 与更广义的鲁棒性

威胁模型与术语区别

Jailbreak

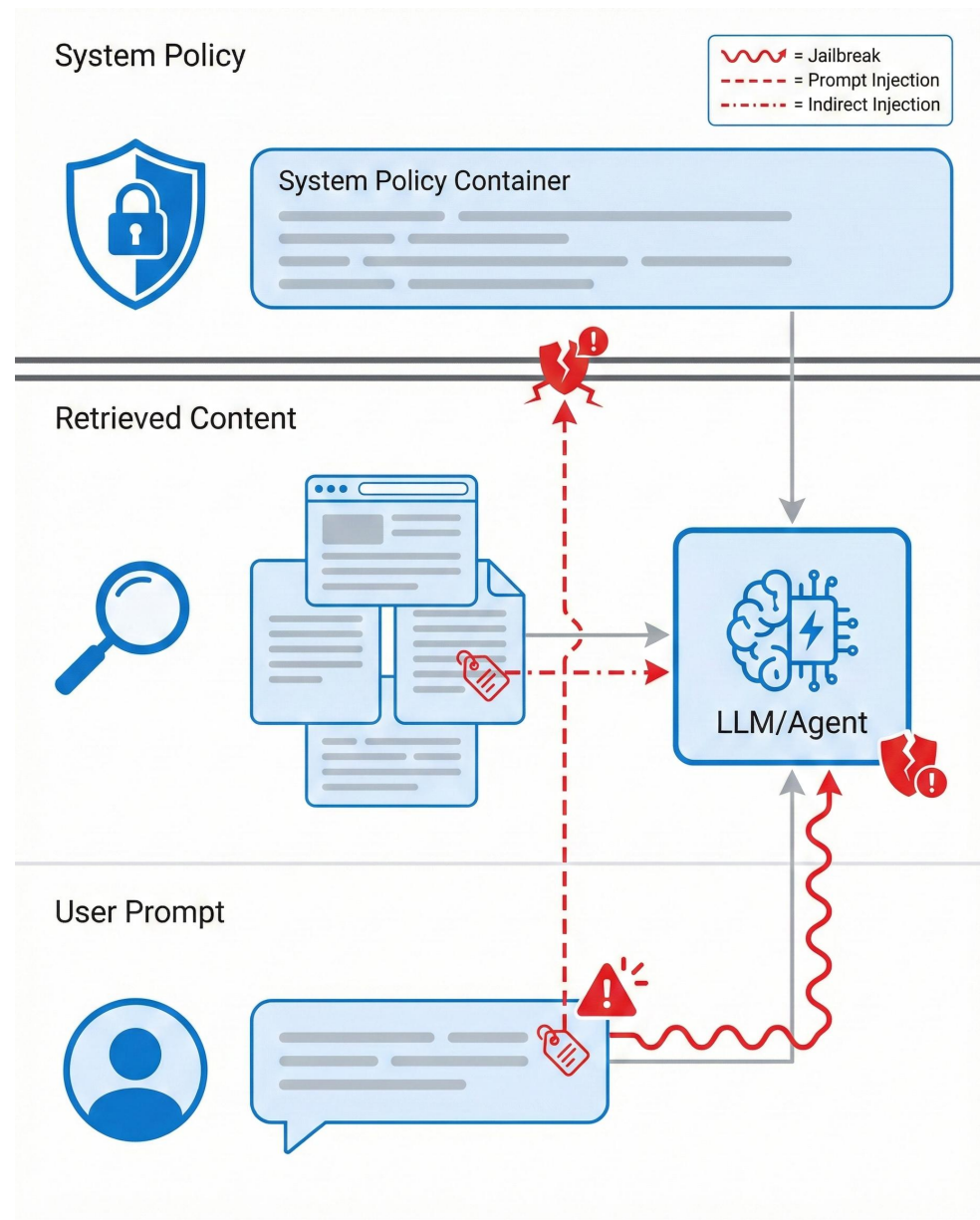
主要是“让模型违背安全策略”的行为（通常不依赖外部文档/工具链）

Prompt Injection

核心是“把恶意指令混入输入，让系统把数据当指令执行”，典型发生在有系统提示词、工具、代理、插件、RAG的场景里

Indirect Injection

恶意指令不在用户显式输入里，而在外部内容（网页、检索文档、邮件、PDF、图片字幕、音频转写等）里，通过系统“摄入”进入上下文



Jailbreak 攻击类型谱系



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

上下文越长，模型越可能被“上下文中的统计规律”牵引
这就是新攻击面

角色扮演

通过设定身份/场景，让模型把原本的安全边界当作“剧情限制可被绕过”的一部分，从而改变拒答倾向

指令分裂

把违规意图拆成多个看似无害的子任务，让模型在组合与推理过程中逐步接近越界输出

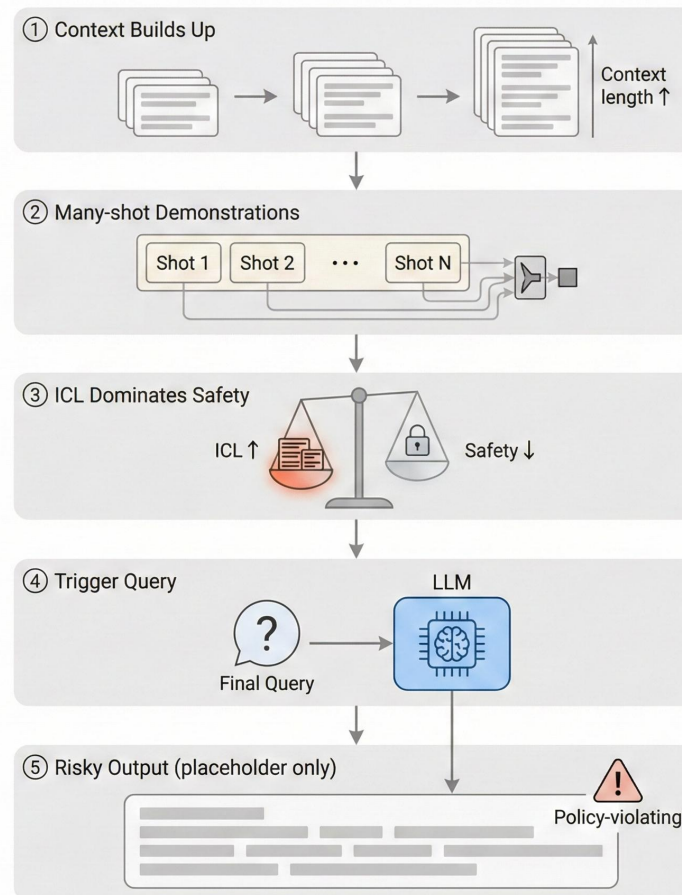
不同的提示结构

多轮累积

利用对话记忆与承诺一致性，让模型在连续回合中逐步抬高越界目标的优先级

编码/混淆

用格式转换与信息遮蔽降低安全过滤的可见度，使模型更可能按“语义意图”而非安全规则来完成任务



总结

在同一对话通道里，通过**提示结构与上下文累积**，改变模型对“应遵循什么指令/约束”的内部判定，导致安全规则**被弱化或被重新解释**

ArtPrompt 与“视觉-文本边界”漏洞 (ASCII Art)



ASCII art (视觉结构)

ASCII art 本质是“文本字符”，但承载信息的方式更像“图像/视觉模式”（靠形状、对齐、空格、布局）

模型没正确解码成语义

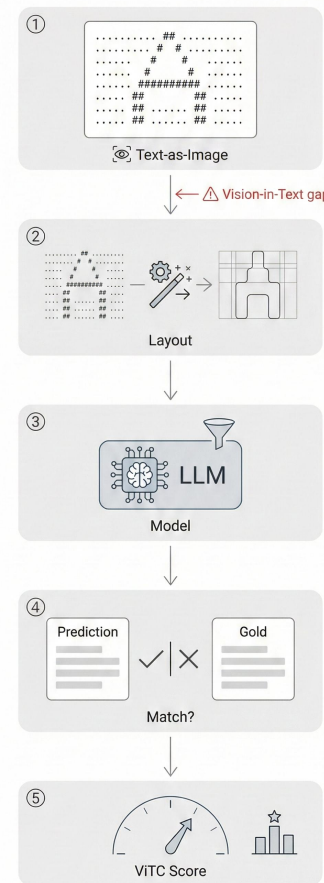
许多 LLM/安全过滤器把文本按 token 线性处理并做语义分类，通常不会先做可靠的版式结构还原，也缺乏对 ASCII art 这类“字符图形”的整体识别能力

语义层安全策略未生效

很多安全对齐与过滤默认“输入主要按语义理解”，对这类“非语义结构信息”识别薄弱，于是过滤没触发或触发不足，出现绕过

出现策略违背输出

最终模型可能生成不符合安全策略的回复，因此在图中用灰色占位块表示“越界输出”而不展示具体内容。



Vision-in-Text Challenge (ViTC)：衡量 LLM 对“不能只靠语义解释、需要视觉/结构还原”的文本输入（如 ASCII art）的识别能力，多种主流模型在这类输入上表现较弱

多模态/外部内容的间接注入（含图片/声音等）

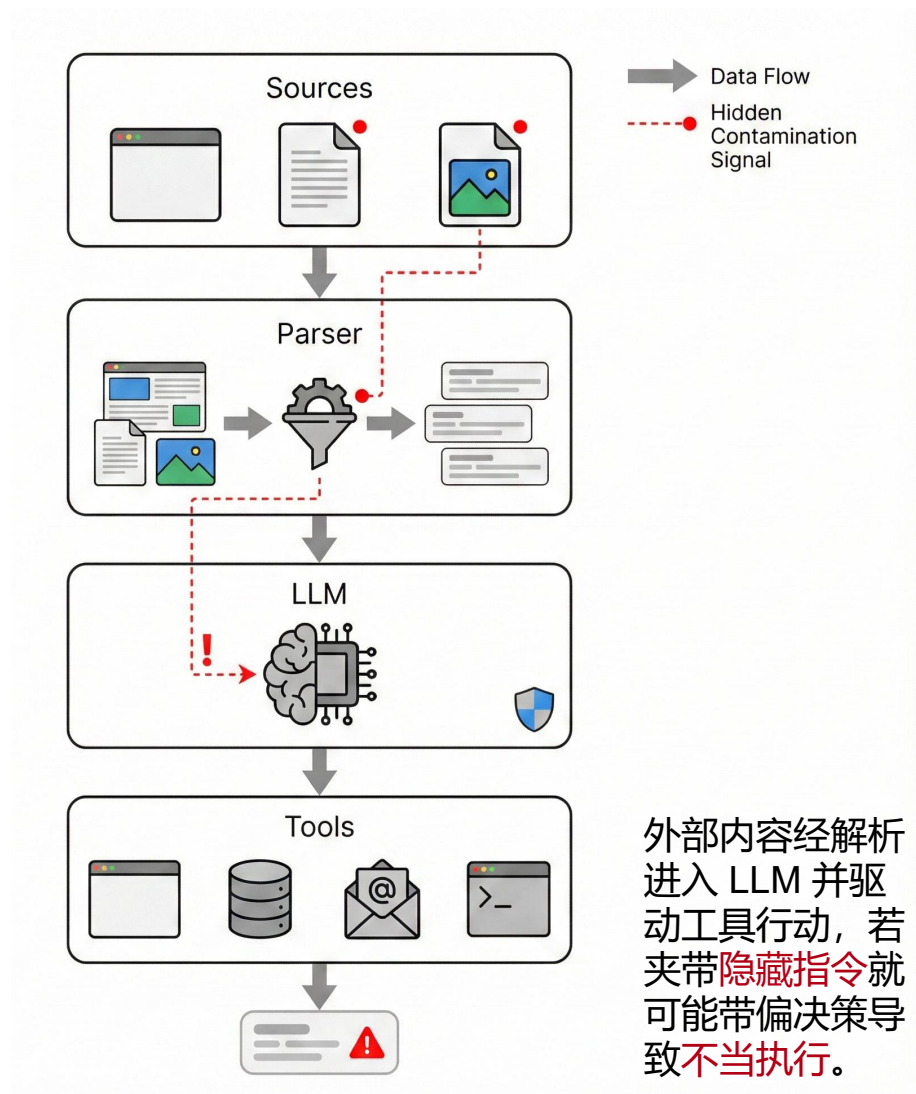
间接注入

- 恶意指令不在用户显式输入中，而是藏在系统会读取的外部内容载体里（文档、网页、图片、音频等）
- 系统把这些内容当“数据/证据”喂给模型时，模型却可能把其中的文本当成“指令”去遵循



- 图片/音频本身不是文本，但系统通常会先经过 OCR/ASR/Caption 等“解析器”把它变成文本或结构化描述，再交给 LLM
- 攻击就利用了这条“解析→拼接→推理/行动”的链路

多模态放大效应



评测与红队：怎么量化“安全鲁棒性”



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

- **JailbreakBench**: 一类专门用来测试“越狱/绕过安全策略”的评测集合或框架。它会提供多种“诱导模型输出不该输出内容”的提示方式, 用来统计模型在不同攻击手法下的表现
- **HarmBench**: 更偏“有害内容生成”的系统化评测(覆盖多种危害类别与指令形式), 常用于衡量模型在“是否会生成明确有害输出”上的整体风险

1) Benchmarks

- **攻击成功率**: 在攻击提示下, 模型“被绕过并输出不该输出内容”的比例
- **拒答一致性**: 面对同类风险请求, 模型拒绝策略是否稳定一致(措辞、边界、规则解释是否漂移)
- **误伤率**: 对本来合规/正常的请求, 模型却过度拒绝的比例
- **多轮稳定性**: 在多轮对话里, 模型是否会被“渐进式诱导”带偏, 出现先拒绝、后松动, 或边界逐轮漂移的情况

2) 评测维度

一个工程化、可迭代的安全流程: **攻击库**→**自动化评测**→**修复**→**回归**

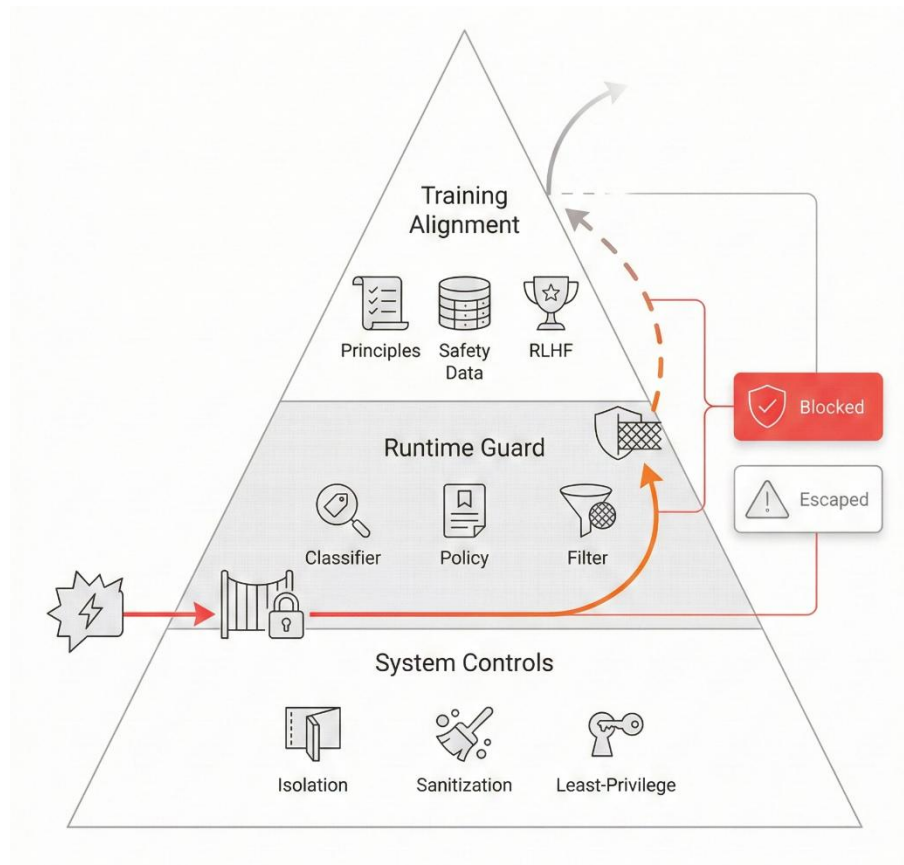
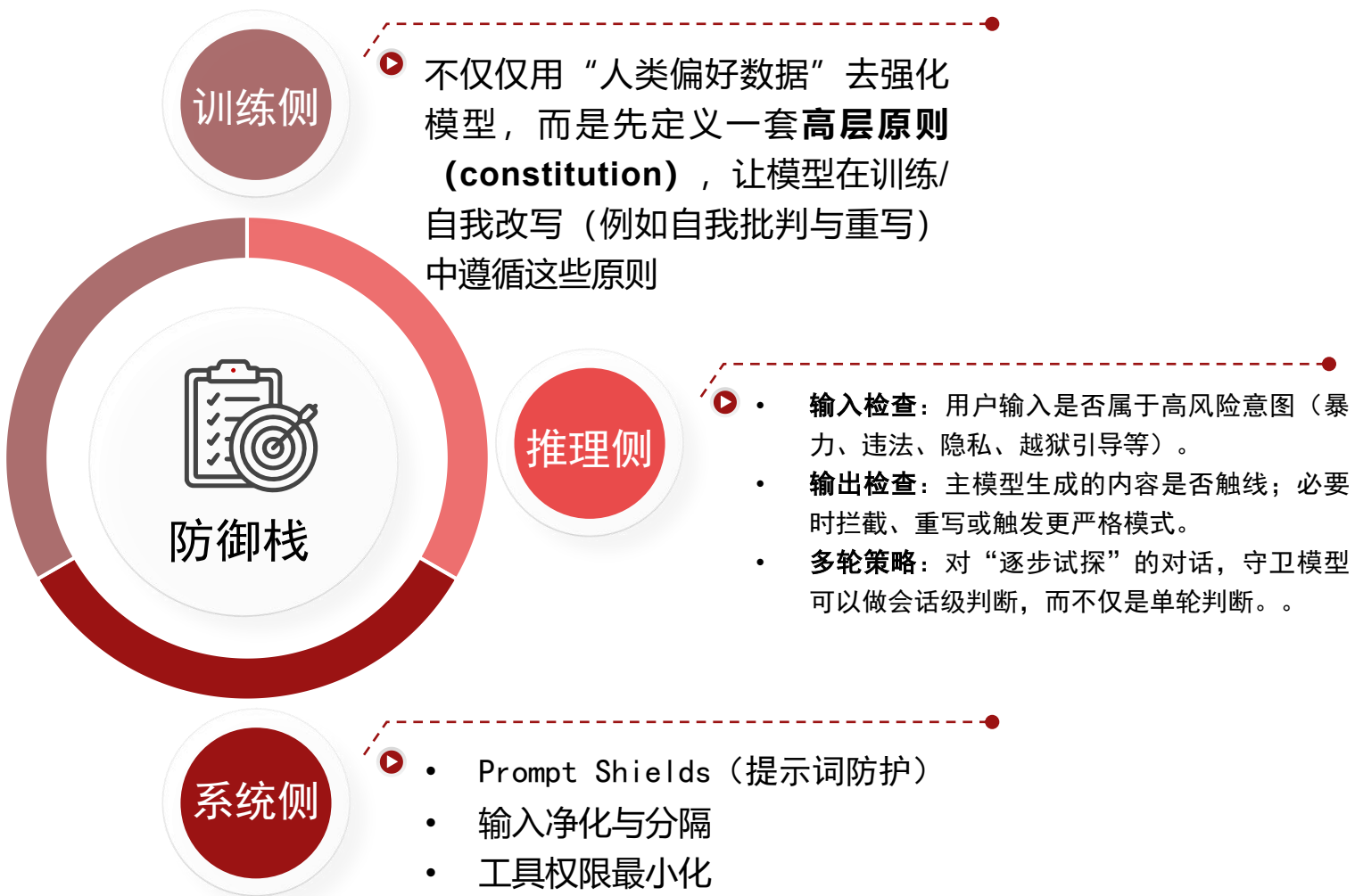
- **攻击库**: 存放“攻击用例集合”(可以是提示策略、场景模板、自动化脚本等)
- **自动化评测**: 用统一脚本跑基准与攻击库, 产出 ASR、误伤率、多轮稳定性等指标
- **修复**: 根据失败案例做改进回归
- **回归**: 把修复后的模型重新跑同一套评测

3) 红队流程

核心

我们怎么用**可重复、可量化**的方法衡量一个大模型到底有多**‘抗越狱’**、多**‘安全稳定’**?

分层防御-防御栈：训练侧 + 推理侧 + 系统侧



核心

工程化防御栈 (defense-in-depth)：不要指望某一个方法（比如只做对齐训练、只上守卫模型、只加系统规则）就能彻底解决越狱与有害输出，而是**分层叠加、相互兜底**



隐写术检测

Steganography vs Watermarking: 攻击与防御的同一“嵌入”范式

Steganography (Attack)

Watermarking (Defense)

人看不出来
(imperceptible)
+系统也难发现
(hard-to-detect)

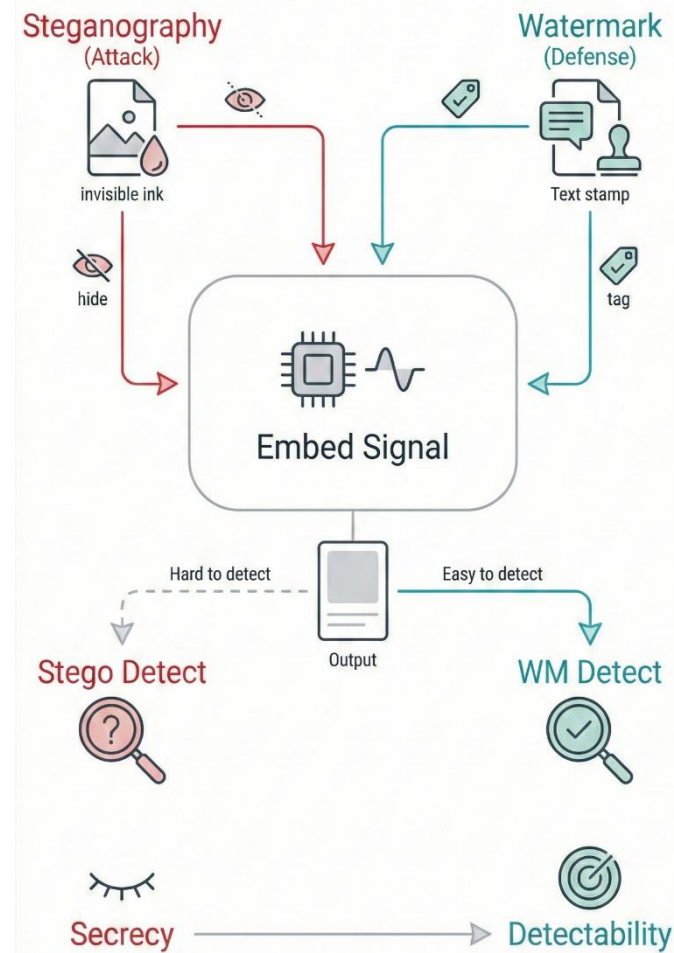
用于偷渡指令

用于数据渗漏/暗通道

让来源标记可检测: AI 生成内容识别+溯源与合规治理

对正常编辑具有一定鲁棒性

检测统计显著+误报/漏报可控



隐写/隐藏注入的检测思路—防护

文本数据

为什么要做、怎么做才算“工程化”

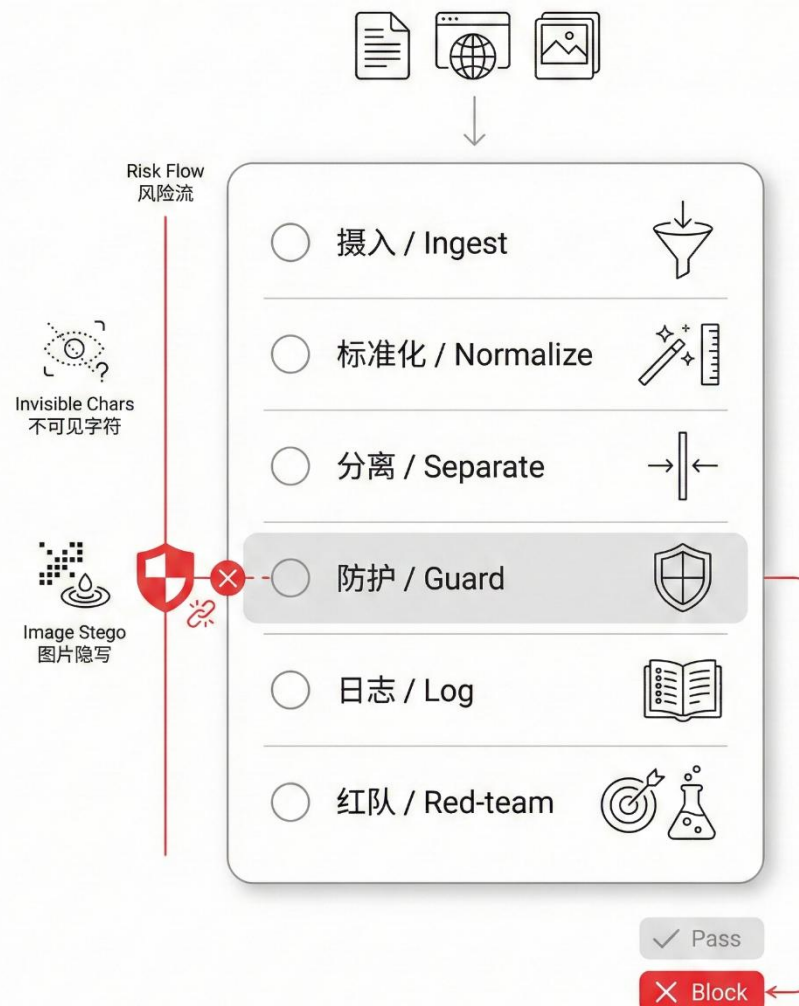
- **Unicode / 不可见字符归一化与净化**
 - 输入统一做 Unicode 归一化、移除/替换不可见字符、对异常字符比例/控制符出现做阈值告警
- **结构化输入的“指令/数据分隔”与策略优先级检查**
 - 将输入分为明确字段、在运行时做策略优先级校验
- **为什么有效**：这本质是“信息流控制”，让模型不把不可信文本当成指令执行。

多模态

图片隐写与 RAG/Agent 摄入链路

- **输入图像的“可疑隐写”检测**（频域/统计特征/模型判别器）
 - 识别图片里可能藏有指令或异常编码（不一定能解码，但能做风险分流
- **对 RAG 摄入内容做安全扫描与隔离执行**
 - 对检索到的网页/文档先做安全扫描
 - 隔离执行
 - 高风险来源触发：只允许“摘要/引用”，不允许“执行型指令”进入决策链

Secure Ingestion Pipeline Checklist



01

不重新训练如何纠错/更新知识（有哪些路线）

02

越狱与注入为何在长上下文/多模态时代更危险

03

隐写/水印：攻击与防御的同一“嵌入”思想